

Contents

1	DD11: 暗号化・鍵管理	1
1.1	0. 文書情報	1
1.2	1. 対象範囲	1
1.3	2. 収録ロジック・対応章	1
1.4	3. 詳細設計本文	1
1.4.1	3.1 Master Key	1
1.4.2	3.2 HKDF-SHA256 派生	2
1.4.3	3.3 AES-256-GCM 列単位暗号化	2
1.4.4	3.4 ローテーション運用	2
1.4.5	3.5 BACKUP_KEY dual-decrypt 期間の運用	3
1.4.6	3.6 PII 列の暗号化対象（参照のみ）	3
1.5	4. 関連設計	4
1.6	5. テスト観点	4
1.6.1	5.1 その他観点	4
1.7	6. 未確定事項・確認事項	4

1 DD11: 暗号化・鍵管理

1.1 0. 文書情報

項目	内容
文書名	DD11: 暗号化・ 管理
詳細設計 ID	DD11
対象システム	FAQ AI ウィジェット SaaS / メインシステム
関連機能 ID	NFR-301（オーナー境界）/ NFR-401（プライバシー保護）/ AC-028
作成日	2026-05-17
版数	v1.0
ステータス	承認済

1.2 1. 対象範囲

種別	ID	名称
ライブラリ	app/shared/src/lib/encrypt.ts	AES-256-GCM + HKDF
シークレット	MASTER_KEY BACKUP_KEY JWT_HS256_KEY 等	管理
カラム	PII 列（email / phone / 氏名 / チャット本文 等）	列単位暗号化
カラム	email_hmac 等	検索用 HMAC 列

1.3 2. 収録ロジック・対応章

元章	元タイトル	概要
§ 10.9.1	Master Key	年次ローテーション運用
§ 10.9.2	HKDF-SHA256 派生	オーナー派生 生成
§ 10.9.3	AES-256-GCM 列単位暗号化	iv + cipher の base64 結合
§ 10.9.4	ローテーション運用	一覧 / dual-decrypt 60 日

1.4 3. 詳細設計本文

1.4.1 3.1 Master Key

MASTER_KEY は Cloudflare Secrets Store に格納（32 bytes、base64）。年次ローテーション運用：

1. 新 MASTER_KEY_NEXT をシークレット追加
2. 双方読み取り対応のコード版をデプロイ（新規書込は新キー、復号は両方試す）

3. バックフィルジョブで既存暗号化列を新キーで再暗号化
4. 完了後 MASTER_KEY_PREV として旧キー保管、MASTER_KEY を新キーに置換
5. 1年後 MASTER_KEY_PREV 削除

1.4.2 3.2 HKDF-SHA256 派生鍵

```
// app/shared/src/lib/encrypt.ts
// contractOwnerUserId は `contract_owners.user_id` (旧オーナー `accounts.id` と物理値同じ ULID)。
// HKDF info 文字列 `owner:<id>` のリテラルは v1.11 リファクタ後も不変 (物理値継承により既存暗号化データ
export async function deriveOwnerKey(masterKeyB64: string, contractOwnerUserId: string): Promise<string> {
  const masterRaw = base64Decode(masterKeyB64);
  const ikm = await crypto.subtle.importKey('raw', masterRaw, 'HKDF', false, ['deriveKey']);
  return crypto.subtle.deriveKey(
    { name: 'HKDF', hash: 'SHA-256',
      salt: new TextEncoder().encode('faq-saas-v1'),
      info: new TextEncoder().encode(`owner:${contractOwnerUserId}`) },
    ikm, { name: 'AES-GCM', length: 256 }, false, ['encrypt', 'decrypt'],
  );
}
```

1.4.3 3.3 AES-256-GCM 列単位暗号化

```
export async function aesGcmEncrypt(key: CryptoKey, plaintext: string): Promise<string> {
  const iv = crypto.getRandomValues(new Uint8Array(12));
  const enc = new TextEncoder().encode(plaintext);
  const cipher = await crypto.subtle.encrypt({ name: 'AES-GCM', iv }, key, enc);
  // 結果は iv(12) + cipher を base64 結合
  return base64Encode(new Uint8Array([...iv, ...new Uint8Array(cipher)]));
}

export async function aesGcmDecrypt(key: CryptoKey, ciphertextB64: string): Promise<string> {
  const raw = base64Decode(ciphertextB64);
  const iv = raw.slice(0, 12);
  const cipher = raw.slice(12);
  const plain = await crypto.subtle.decrypt({ name: 'AES-GCM', iv }, key, cipher);
  return new TextDecoder().decode(plain);
}
```

1.4.4 3.4 鍵ローテーション運用

オーナー派生 は MASTER_KEY のローテーションで自動的に変わる（派生関数の入力が変わるため）。再暗号化バックフィルが必須。

1.4.4.1 3.4.1 鍵ローテーション一覧 (両書同期) 顧管 § 12.6 / § 12.6.1 の表現と揃える。MVP では以下の を 年次ローテーション + 60 日 dual-decrypt 期間で運用する。

鍵 / シークレット	用途	HKDF info 値	ローテーション周期	dual-decrypt 期間	主管
MASTER_KEY	オーナー派生 の HKDF 元	(HKDF info=owner:<id>)	年次	60 日 (旧 MASTER_KEY_PREV 並走)	メイン
派生 audit-export	監査ログエクスポート暗号化	audit-export	MASTER_KEY 連動	同上	顧管 § 12.6
派生 mfa-setup	MFA 初期化トークン HMAC	mfa-setup	MASTER_KEY 連動	同上	顧管
派生 password-reset	パスワードリセット HMAC	password-reset	MASTER_KEY 連動	同上	共通
派生 re-auth	再認証セッション HMAC	re-auth	MASTER_KEY 連動	同上	共通

鍵 / シークレット	用途	HKDF info 値	ローテーション周期	dual-decrypt 期間	主管
派生 internal-api (JWT_HS256_KEY)	連携 IF #1～#12 JWT HS256 署名	internal-api	年次 + 60 日 dual-decrypt (顧管 §12.6.2 と同形式)	60 日	メイン / 顧管 共通
BACKUP_KEY	R2 バックアップ暗号化 (AES-256-GCM)	backup-key	年次 + 60 日 dual-decrypt	60 日	共通
mTLS Origin CA 証明書	連携 IF #1～#12 クライアント証明書	—	年次	30 日	Cloudflare 管理

重要: JWT_HS256_KEY のローテーション運用は顧管 §12.6.2 と整合させ、本書(メイン側)でも **60 日 dual-decrypt** 期間を設ける。これにより、メイン側 Worker が新 で発行した JWT を顧管側 Worker が旧 で受信した場合(デプロイのタイムラグ)でも 60 日間は検証成功する。

1.4.4.2 3.4.2 dual-decrypt 期間の運用

- ・ **鍵更新フロー:** (1) 新 JWT_HS256_KEY_NEXT を wrangler secret put で投入、(2) 検証側 Worker (本書側 internal-api + 顧管側 admin-api) を **dual-decrypt 対応版** にデプロイ、(3) 発行側 Worker (メイン側で IF #11 / IF #12 を発火、顧管側で IF #1～#7 を発火) を **新鍵で発行する版** にデプロイ、(4) 60 日経過後、旧 JWT_HS256_KEY_PREV を削除。
- ・ **検証実装** (両書共通):

```
async function verifyJwt(env: Env, token: string) {
  const keys = [env.JWT_HS256_KEY, env.JWT_HS256_KEY_PREV].filter(Boolean);
  for (const k of keys) {
    if (await verifyWithKey(token, k)) return true;
  }
  throw new HTTPException(401, { message: 'INVALID_JWT' });
}
```

- ・ **監査ログ:** 更新時は key.rotation.start / key.rotation.complete (5y / operator_high_priv) を audit_logs に記録。
- ・ **アラート:** 旧 (_PREV) の使用率が 0% に近づいたら 60 日経過前でも撤去判断 (Cloudflare Analytics で旧使用ヒット数を監視)。

1.4.5 3.5 BACKUP_KEY dual-decrypt 期間の運用

- ・ **更新フロー:** (1) 新 BACKUP_KEY_NEXT を wrangler secret put で投入、(2) バックアップ書込ジョブを **BACKUP_KEY** で書き続け、**BACKUP_KEY_NEXT** で並行書込 (60 日間 dual-write 相当)、(3) 60 日経過後、新を BACKUP_KEY に rename、旧を BACKUP_KEY_PREV に退避、(4) さらに 60 日経過 (= 古いバックアップが期限切れ) で BACKUP_KEY_PREV を削除。
- ・ **復号時:** バックアップオブジェクトのメタデータに key_version を持たせ、復号時は key_version に対応する で AES-256-GCM 復号。

// 概念実装

```
async function decryptBackup(env: Env, r2object: R2Object): Promise<Buffer> {
  const keyVersion = r2object.customMetadata?.keyVersion ?? 'v1';
  const key = keyVersion === 'v_current' ? env.BACKUP_KEY
    : keyVersion === 'v_previous' ? env.BACKUP_KEY_PREV
    : null;
  if (!key) throw new Error('UNKNOWN_BACKUP_KEY_VERSION');
  return aesGcmDecrypt(key, await r2object.arrayBuffer());
}
```

- ・ **アラート:** 古い BACKUP_KEY_PREV の使用率が 0% に近づいたら 60 日経過前でも早期撤去判断。

1.4.6 3.6 PII 列の暗号化対象 (参照のみ)

PII 暗号化対象列の全リスト (email / phone / 氏名 / チャット本文 等) は [./02_基本設計/03_テーブル設計.md](#) を正本とする。各列は AES-256-GCM 暗号化値 (base64) で保存し、検索用に HMAC-SHA256 列 (email_hmac 等) を併設する。

1.5 4. 関連設計

種別	参照先
要件	../01_要件定義/index.md
基本設計	../02_基本設計/index.md
セキュリティ設計	../02_基本設計/09_セキュリティ設計.md
テーブル設計	../02_基本設計/03_テーブル設計.md
共有概念	../共有/共有概念.md
運用設計	../04_運用設計/index.md
将来対応	../05_future/index.md
関連 DD	DD08_トークン発行・検証.md / DD10_監査ログ書込・完全性検証.md / DD14_バッチ・非同期処理.md

1.6 5. テスト観点

AC ID	テスト ID	テスト方式	テストファイル
AC-028	audit-privacy-001	Manual audit	docs/audit/privacy/YYYY-MM-DD.md

1.6.1 5.1 その他観点

観点	内容
単体	aesGcmEncrypt / aesGcmDecrypt の往復一致 / deriveOwnerKey のオーナー違い検証
結合	ローテーション中 (dual-decrypt 期間) の新旧両キーでの復号成功
異常系	BACKUP_KEY_PREV 削除後のオブジェクト復号時 UNKNOWN_BACKUP_KEY_VERSION
セキュリティ	同じ平文 + 別 iv で暗号文が異なる (IND-CPA 性質) / Authentication Tag 改ざん検知
ローテーション	年次バックアップジョブで全 PII 列の再暗号化が完了
連携 IF	メイン → 顧客の JWT (HS256) を 60 日 dual-decrypt 期間中に旧 / 新 で送受信

1.7 6. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済